

Lab 24: Locally Weighted Regression (LWR)

Aim

To implement **Locally Weighted Regression (LWR)** to predict continuous output values by giving more importance to training points that are closer to the query point.

Algorithm

1. Import the required libraries.
2. Create a sample dataset.
3. Define the query point for prediction.
4. Calculate the distance between each training point and the query point.
5. Assign weights using a Gaussian function.
6. Perform weighted linear regression.
7. Predict the output value.
8. Display the predicted value.

Program

```
# Lab 24: Locally Weighted Regression (LWR)
```

```
import numpy as np
```

```
# Sample dataset
```

```
X = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
```

```
y = np.array([2, 4, 5, 4, 5, 7, 8, 9], dtype=float)
```

```
# Query point
```

```
query = 5.5
```

```
# Bandwidth parameter
```

```

tau = 1.0

# Calculate weights using Gaussian function
weights = np.exp(-((X - query) ** 2) / (2 * tau ** 2))

# Add bias term
X_design = np.column_stack((np.ones(len(X)), X))

# Create weight matrix
W = np.diag(weights)

# Calculate weighted regression parameters
theta = np.linalg.pinv(
    X_design.T @ W @ X_design
) @ X_design.T @ W @ y

# Predict for query point
query_design = np.array([1, query])

prediction = query_design @ theta

print("Query Point:", query)
print("Predicted Value:", round(prediction, 3))

```

Sample Output

```

Query Point: 5.5
Predicted Value: 6.047

```

Result

Thus, the **Locally Weighted Regression algorithm** was successfully implemented, and the output value for the given query point was predicted by assigning higher weights to nearby data points.